

Java Collection Framework - Detailed Notes

1. Introduction to Java Collection Framework

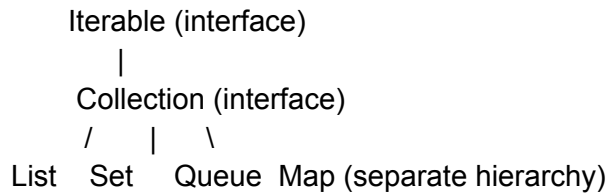
The **Java Collection Framework (JCF)** is a set of classes and interfaces that provide a standardized way to store, retrieve, and manipulate collections of objects. It was introduced in **Java 2 (JDK 1.2)** to replace legacy classes like `Vector` and `Hashtable`.

Key Benefits of Collection Framework:

- **Reusable and efficient:** Provides ready-made data structures and algorithms.
 - **Consistency:** Implements standard interfaces like `Collection` and `Map`.
 - **Scalability:** Supports large amounts of data with optimized performance.
 - **Flexibility:** Provides different data structures like Lists, Sets, and Queues.
 - **Thread-Safety:** Some implementations like `ConcurrentHashMap` provide thread-safe operations.
-

2. Hierarchy of Java Collection Framework

The Collection Framework consists of multiple interfaces and classes. The main hierarchy is:



Major Interfaces in Java Collection Framework

Interface	Description
Collection	Root interface of the collection framework.
List	Ordered collection, allows duplicates (e.g., ArrayList , LinkedList).
Set	Unordered collection, does not allow duplicates (e.g., HashSet , TreeSet).
Queue	Follows FIFO (First In, First Out) order (e.g., PriorityQueue).
Deque	Double-ended queue, allows insertion/removal from both ends (e.g., ArrayDeque).
Map	Stores key-value pairs (e.g., HashMap , TreeMap).

3. List Interface

A **List** is an ordered collection that allows duplicate elements.

Implementations of List:

1. **ArrayList** (Resizable array implementation, fast random access)
2. **LinkedList** (Doubly linked list, efficient insertion/deletion)
3. **Vector** (Thread-safe, legacy class)
4. **Stack** (LIFO structure, extends Vector)

Example:

```
import java.util.*;
public class ListExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Apple");
        System.out.println(list); // Output: [Apple, Banana, Apple]
    }
}
```

4. Set Interface

A **Set** is an unordered collection that does not allow duplicate elements.

Implementations of Set:

1. **HashSet** (Unordered, fast access using hashing)
2. **LinkedHashSet** (Maintains insertion order)
3. **TreeSet** (Sorted set using Red-Black tree)

Example:

```
Set<Integer> set = new HashSet<>();  
set.add(10);  
set.add(20);  
set.add(10);  
System.out.println(set); // Output: [10, 20]
```

5. Queue Interface

A **Queue** follows FIFO (First In, First Out) order.

Implementations of Queue:

1. **PriorityQueue** (Elements are ordered based on priority)
2. **ArrayDeque** (Efficient double-ended queue operations)

Example:

```
Queue<Integer> queue = new LinkedList<>();  
queue.add(10);  
queue.add(20);  
queue.poll(); // Removes 10  
System.out.println(queue); // Output: [20]
```

6. Map Interface

A **Map** stores key-value pairs and does not allow duplicate keys.

Implementations of Map:

1. **HashMap** (Unordered key-value storage)
2. **LinkedHashMap** (Maintains insertion order)
3. **TreeMap** (Sorted map based on keys)
4. **Hashtable** (Thread-safe, legacy class)

Example:

```
Map<String, Integer> map = new HashMap<>();
map.put("A", 1);
map.put("B", 2);
map.put("A", 3); // Overwrites key 'A'
System.out.println(map); // Output: {A=3, B=2}
```

7. Differences Between List, Set, and Map

Feature	List	Set	Map
Allows Duplicates	Yes	No	No (keys)
Maintains Order	Yes	No	No (except LinkedHashMap)
Access by Index	Yes	No	No
Key-Value Pairs	No	No	Yes

8. Sorting in Collections

Using `Collections.sort()` for Lists

```
List<Integer> list = new ArrayList<>(Arrays.asList(5, 2, 9, 1));
Collections.sort(list);
System.out.println(list); // Output: [1, 2, 5, 9]
```

Using `TreeSet` for Sorted Set

```
Set<String> treeSet = new TreeSet<>();
treeSet.add("Banana");
treeSet.add("Apple");
System.out.println(treeSet); // Output: [Apple, Banana]
```

9. Thread-Safe Collections

Java provides thread-safe collections under `java.util.concurrent` package.

Thread-Safe Collection	Alternative
<code>Vector</code>	<code>ArrayList</code> with <code>Collections.synchronizedList()</code>
<code>Hashtable</code>	<code>ConcurrentHashMap</code>
<code>Stack</code>	<code>Deque</code> (<code>ArrayDeque</code>)

Example:

```
List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());
syncList.add(10);
syncList.add(20);
```

10. Summary

- **List** allows duplicates and maintains order.
- **Set** does not allow duplicates.
- **Queue** follows FIFO order.
- **Map** stores key-value pairs.
- Use `Collections.sort()` for sorting lists.
- Use `ConcurrentHashMap` for thread-safe operations.

The Java Collection Framework is an essential part of Java programming, making data manipulation efficient and standardized.